

Virtual Teammate Portal — Developer Reference

Architecture and internals of the staff/client portal under `/portal/`. For the end-user walkthrough, see `portal-user-guide.md`.

Scope note: the PWA / installable-app and Web Push stack (`manifest.webmanifest` , `sw.js` , `offline.html` , `assets/pwa.js` , `assets/push.js` , `webpush.php` , and the `push_subscriptions` table) is **intentionally out of scope** for this document and will be documented separately. Where `notify()` fans out to push, that branch is noted but not detailed here.

1. Stack

- **PHP 8.2**, `declare(strict_types=1)` throughout. **No external dependencies** (no Composer, no PHPMailer) — everything is hand-rolled.

- **SQLite** via PDO (prepared statements everywhere).
- Served by the same Apache instance as the marketing site, at `/portal/`.

2. Architecture

Bootstrap — `portal/bootstrap.php`

Loaded by every entry point. Responsibilities:

- Strict error handling; secure session init (httponly, `samesite=Lax` , `use_strict_mode` , `use_only_cookies` , session name `vtportal` , custom save path `data/sessions/`).
- HTTPS enforcement (toggleable by super admin; state shared with the marketing site).
- **DB access:** `db()` and `chatdb()` return singleton PDOs (main DB + isolated chat DB).
- **Auth helpers:** `current_user()` (cached per request), `require_login()` , `require_role(...)` , `is_super_admin()` .
- **CSRF:** `csrf_token()` , `csrf_field()` , `csrf_verify()` .
- **Rendering:** `render($view, $vars)` , `redirect()` , `flash()` .
- **Media:** `tbl_thumb()` , `media_src()` , `media_thumb_src()` .
- **Audit:** `audit_log()` .

- **Notifications/email:** `notify()` (DB row + optional email + push), settings cache via

`get_setting()` / `set_setting()`.

- Escaping helper `e()` (`htmlspecialchars`) used in views.

Front controller — `portal/index.php`

A single switch dispatches on a query parameter: `?p=<action>`. There are ~96 handlers, each a function with the signature:

```
function handle_<action>(): void { ... }
```

Handler conventions:

- GET handlers query the DB and end with `render('<view>', $data)`.
- POST handlers call `csrf_verify()` first, mutate state, then `audit_log()` + `flash()` + `redirect()` (post/redirect/get).
- Every handler calls `require_login()` (and usually `require_role(...)`) before doing work.

Views — `portal/views/`

`render()` loads a view from `views/` and (by default) wraps it in `views/layout.php`, which provides the role-aware sidebar nav, topbar, and flash messages. Unauthenticated/standalone pages (login, install, error) pass `'_naked' => true` to skip the chrome.

3. Database

- **Main DB:** `data/portal.sqlite` (auto-created by `install.php`).
- **Chat DB:** `data/chat.sqlite` — messaging lives in its own database so high-frequency

chat polling/writes never lock the main DB.

- **Schema:** `portal/schema.sql` (all `CREATE TABLE IF NOT EXISTS`, idempotent).
- **Pragmas:** WAL journal mode, `busy_timeout=15000`, `synchronous=NORMAL`, `foreign_keys=ON`.

Table reference

Table	Purpose
<code>users</code>	All accounts (super_admin / client / csm / vt_hired / vt_onpool). Auth, profile, HubSpot ids, <code>vt_status</code> , <code>active</code> , <code>last_login_at</code> , <code>notify_by_email</code> .
<code>clients</code>	Client companies linked to a user; contract status; HubSpot company/owner ids.
<code>vt_profiles</code>	Extended VT data: status, department, role title, experience, English level, skills, resume/video URLs, tracker id.
<code>vt_profile_meta</code>	One row per synced HubSpot property per VT (audit trail of the mapping).
<code>company_profiles</code>	Extended client company data (website, industry, size, address).
<code>csm_clients</code>	M2M: which CSMs manage which clients.
<code>client_vts</code>	M2M: which VTs are assigned to which clients (with contract status, start/end).
<code>meetings</code>	Calendar events (organizer, attendee/role, time, link, topic, status).
<code>meeting_attendees</code>	M2M: multiple attendees per meeting.
<code>eod_reports</code>	VT end-of-day reports (best work, help needed, focus next, KPI).
<code>tasks</code>	Assignable to-dos (priority, status, due date, completion).
<code>task_attachments</code>	Files per task (stored on disk under <code>data/task-attachments/{task_id}/</code>).
<code>messages</code>	1:1 chat (canonical <code>conversation_key</code> , read receipts) — lives in <code>chat.sqlite</code> .
<code>notifications</code>	Per-user alerts (kind: task/meeting/message/sync/info; read state; link).
<code>workday_logs</code>	VT work-day summaries (start/end, minutes, client context).
<code>audit_log</code>	Immutable record of actions (actor, action, entity, details, IP).
<code>app_settings</code>	Key/value config (HubSpot token, sync state, email settings, etc.).
<code>traffic</code>	Marketing-site pageview beacons (path, IP, geo, UA, referrer).
<code>geo_cache</code>	IP → geolocation cache.
<code>leads</code>	Lead-form submissions captured by the marketing site's <code>/lead.php</code> .
<code>push_subscriptions</code>	Web Push device endpoints. <i>(Out of scope — see the PWA doc.)</i>

Migrations — `portal/install.php`

One-shot installer + migration runner. Idempotent: re-creates tables `IF NOT EXISTS`, rebuilds the `users.role` CHECK constraint to drop legacy states, backfills added columns (`hubspot_contact_id`, `notify_by_email`, ...), runs one-time data migrations, and creates indices on foreign-key columns (important for fast cascade deletes during HubSpot purges).

4. Auth & sessions

Login (`?p=login`, POST):

1. `csrf_verify()`.
 2. `SELECT ... FROM users WHERE email = :email AND active = 1`.
 3. `password_verify($pw, $row['password_hash'])` (bcrypt).
 4. On success: `session_regenerate_id(true)`, set `$_SESSION['uid']`, regenerate the CSRF token, `UPDATE users SET last_login_at = ...`, `audit_log('login', ...)`, redirect to `next` or the dashboard.
1. On failure: `audit_log('login_failed', ...)` and show an error.

Session hardening: strict mode, httponly, cookies-only, `samesite=Lax`, custom save path (`data/sessions/`, so it doesn't depend on the host `/tmp`).

CSRF: token in `$_SESSION['csrf']`; forms include `csrf_field()`; POSTs verify with constant-time `hash_equals()`.

Permissions: `require_login()` redirects to login (preserving `next`); `require_role(...$roles)` renders a 403 view if the current user's role isn't allowed. Soft-delete = set `users.active = 0`.

Seeded passwords: records created by a HubSpot sync get a temporary default password by role (e.g. `client12345`, `csm12345`, `vt_hired12345`, `vt_onpool12345`) set **only on insert** — existing users keep their password. These are intended to be changed on first login.

5. Roles & permission enforcement

Roles are an enum on `users.role`: `super_admin`, `client`, `csm`, `vt_hired`, `vt_onpool`. Enforcement happens per handler via `require_role(...)`, and the sidebar in `views/layout.php` is role-aware so each role only sees its allowed nav. Super admin bypasses most role gates (e.g. can view any user's messages). `vt_onpool` gets a trimmed nav (no assignments/meetings/payslips/CSM relationship) compared to `vt_hired`.

6. HubSpot sync state machine — `portal/hubspot.php`

A batched, resumable sync (~1k lines) so no single request times out. Two independent pipelines: **talent** (VTs) and **clients** (companies + CSMs). State is persisted in `app_settings`; the browser polls a step endpoint until the state reports `done`. Runtime limits are lifted during a run (`set_time_limit(0)`, raised memory and socket timeout).

States:

1. **search** — `POST /crm/v3/objects/contacts/search` with a single equality filter (e.g.

`hs_lead_status = "Virtual Teammate"`), paginated via `paging.next.after` (up to 100/page).

1. **process** — batch ~20 records per step. `hs_find_user_for_contact()` upserts by HubSpot

contact id, then by email. Maps HubSpot properties → portal columns and maps status → role (`Hired` → `vt_hired`; `Matched` / `Unmatched` → `vt_onpool`; ineligible → delete). Every pulled property is recorded in `vt_profile_meta`.

1. **media** — download photos/resumes/videos concurrently (`curl_multi`, ~4 in flight). Strips

Google-Drive size qualifiers; recognizes hosted video (YouTube/Vimeo/Loom) and stores the URL as-is. Saves to `data/media/vt/{id}/` (gated) and public photos to `vtmedia/vt/{id}/`, generating a 150×150 webp thumbnail at `vtmedia/vt_thumbs/{id}.webp`. Checkpoints per file so a killed batch resumes.

1. **done** — checkpoint timestamp; polling stops.

The client pipeline is analogous: search companies → create `clients` + `company_profiles` → create/refresh the CSM contact (`users.role = csm`) → link via `csm_clients`.

Key handlers: `hubspot` (page), `hubspot.save_settings`, `hubspot.test`, `hubspot.control` (pause/resume/reset), `hubspot.step`, the dedicated `hubspot.talent_` / `hubspot.client_` control/step/search/sync_one variants, `hubspot.purge` / `hubspot.purge_all`, `hubspot.seed_demo`.

Settings (`app_settings`): `hs_token`, the VT/client/CSM lead-status field+value filters, `hs_vt_status_field`, `hs_batch_size`, `hs_import_media`, and the running `hs_sync_state`.

TLS: a CA bundle is located from the PHP/OpenSSL configuration (`.../extras/ssl/cacert.pem`) with the shipped `portal/cacert.pem` as fallback, used for HubSpot API calls.

7. Notifications & email

`notify(int $userId, string $kind, string $title, string $body, string $link):`

1. Inserts a row into `notifications`.
2. If `users.notify_by_email = 1`, sends a branded email.

3. Fans out to the user's push subscriptions. *(Push mechanics are out of scope here — see the PWA doc.)*

All three are best-effort and isolated: a failure never crashes the request.

Email transport. If `portal/smtp.local.php` is configured (Google Workspace; needs an app password), a minimal hand-rolled client does STARTTLS + AUTH LOGIN on port 587. Otherwise it falls back to native `mail()` (available on the production server). Messages are wrapped in `portal_email_shell()` — a gradient-header branded HTML shell with a text-only logo (no remote image fetch, so it renders reliably). `smtp.local.php` is server-only and never deployed.

Email composer (super admin): `email` (compose), `email.send` (send via the same transport + audit), `email.settings` (set the address that receives new-lead notifications).

8. Lead capture integration

The marketing site's `/lead.php` writes directly into this portal's `leads` table (same SQLite file). On the portal side:

- `leads` — lists submissions (with a "new since last viewed" badge tracked via an `app_settings` timestamp).
- `leads.delete` — removes a lead.
- New submissions also trigger a notification email to the configured lead-notify address.

Super-admin only.

9. Folder structure & file protection

```
portal/
├─ index.php           # front controller (~96 handlers)
├─ bootstrap.php       # sessions, DB, auth, CSRF, render, notify, settings
├─ hubspot.php         # sync state machine + media import
├─ schema.sql          # SQLite DDL (idempotent)
├─ install.php         # installer + migration runner
├─ install.key.php(.example) # gitignored secret for remote first-run
├─ smtp.local.php(.example) # Google Workspace SMTP config (server-only)
├─ cacert.pem          # CA bundle fallback for outbound TLS
├─ assets/             # portal.css, portal.js (+ PWA assets – out of scope here)
└─ views/              # ~50 templates: layout.php, dashboards per role,
                        #   *-list/*-edit/*-view CRUD, meetings, eod, tasks,
                        #   messages, notifications, leads, funnel, hubspot,
                        #   email-compose, audit, traffic, my-vts/my-clients/my-team,
                        #   productivity, resources, payslips, invoices, refer, etc.

data/                  # runtime, web-blocked, not deployed
├─ portal.sqlite       # main DB
├─ chat.sqlite         # isolated messaging DB
├─ sessions/          # PHP session files
├─ task-attachments/{task_id}/
├─ media/vt/{id}/      # gated resumes/videos
└─ SUPERADMIN_CREDENTIALS.txt # generated on install (web-blocked)

vtmedia/               # web-accessible public talent media
└─ vt/{id}/ , vt_thumbs/{id}.webp
```

Protection. A `.htaccess` blocks web access to `data/`, `install.key.php`, and `SUPERADMIN_CREDENTIALS.txt`. `/portal/` and `/vtmedia/` are web-accessible; gated media is served through PHP handlers (`avatar`, `media`) that enforce auth, not by direct URL.

10. Conventions & safeguards

- PDO prepared statements only (no string-built SQL); `e()` for output escaping (XSS).
- `current_user()` and settings are cached per request.
- Audit log is append-only and records actor, action, entity, details, and IP for every

state-changing action.

- Best-effort side effects (notifications, email, audit, push) are wrapped so they can never break the primary request.
- Chat is isolated in its own DB; foreign-key indices keep cascade deletes (especially HubSpot purges) fast.